Trie & String Algorithms

> → **Key Idea**: Use prefix trees and pattern matching for efficient string operations

> • **When to use**: Prefix matching, autocomplete, pattern searching, dictionary operations

> ⊕ **Complexity**: Trie operations O(m) where m is string length, KMP O(n+m)

> ⚠ **Common Pitfall**: Not handling memory efficiently in trie or implementing pattern matching incorrectly

Overview

Visual Example

Trie Structure Visualization

```
Words: ["cat", "car", "card", "dog", "dodge"]

Trie Structure:
                root
               /    \
              c      d
              |      |
              a      o
              |      |
              t      g
             /|\    /|\
            ★ r e  ★ d
              |      |
              ★      g
              |      |
              d      e
              |      |
              ★      ★

★ = End of word marker

>> Prefix "ca" shared by "cat", "car", "card"
>> Prefix "dog" shared by "dog", "dodge"
```

Trie Search Process

```
Search for "car":

Step 1: Start at root
Step 2: Follow 'c' → found
Step 3: Follow 'a' → found
Step 4: Follow 'r' → found + end marker ★
Result: "car" exists in trie ✓

Search for "care":
Step 1-3: Same as above
Step 4: Follow 'e' → not found ✗
Result: "care" does not exist

>> Search complexity: O(length of word)
```

KMP Failure Function

```
Pattern: "ABABCABAB"
Text:    "ABABDABABCABABCABAB"

Build failure function for pattern:
A B A B C A B A B
0 0 1 2 0 1 2 3 4
│ │ │ │ │ │ │ │ │
└─┼─┘ │ │ └─┼─┘ │
  └───┘ │   └───┘
        │
   No prefix match

KMP Search with failure function:
Text:    A B A B D A B A B C A B A B C A B A B
Pattern: A B A B C A B A B
         ↑ ↑ ↑ ↑ ✗
         Use failure function to skip ahead instead of restarting

⚡ Avoids redundant character comparisons
```

Interview Tips

Pattern Recognition

- When to identify this technique in problem statements
- Key indicators and problem characteristics
- Similar problem types and variations

Common Mistakes

- Implementation pitfalls to avoid

- Edge cases that are often missed
- Performance considerations

Implementation Variations

```
// Alternative approaches and optimizations
// Template code for common variations
```

Common Problem Patterns

1. Implement Trie (Prefix Tree)

Problem: Implement a trie with insert, search, and startsWith.

```
class Trie {
    private TrieNode root = new TrieNode();

    class TrieNode {
        TrieNode[] children = new TrieNode[26];
        boolean isEnd = false;
    }

    public void insert(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            if (node.children[c - 'a'] == null) node.children[c - 'a'] = new TrieNod
            node = node.children[c - 'a'];
        }
        node.isEnd = true;
    }

    public boolean search(String word) {
        TrieNode node = searchPrefix(word);
        return node != null && node.isEnd;
    }

    public boolean startsWith(String prefix) {
        return searchPrefix(prefix) != null;
    }

    private TrieNode searchPrefix(String prefix) {
        TrieNode node = root;
        for (char c : prefix.toCharArray()) {
            if (node.children[c - 'a'] == null) return null;
            node = node.children[c - 'a'];
        }
        return node;
```

```
    }
}
```

2. Word Search II

Problem: Find all words from dictionary that exist in board.

```
public List<String> findWords(char[][] board, String[] words) {
    List<String> result = new ArrayList<>();
    TrieNode root = buildTrie(words);

    for (int i = 0; i < board.length; i++) {
        for (int j = 0; j < board[0].length; j++) {
            dfs(board, i, j, root, result);
        }
    }
    return result;
}

private void dfs(char[][] board, int i, int j, TrieNode node, List<String> result) {
    char c = board[i][j];
    if (c == '#' || node.children[c - 'a'] == null) return;
    node = node.children[c - 'a'];
    if (node.word != null) {
        result.add(node.word);
        node.word = null; // Avoid duplicates
    }

    board[i][j] = '#';
    if (i > 0) dfs(board, i-1, j, node, result);
    if (j > 0) dfs(board, i, j-1, node, result);
    if (i < board.length-1) dfs(board, i+1, j, node, result);
    if (j < board[0].length-1) dfs(board, i, j+1, node, result);
    board[i][j] = c;
}
```

3. Longest Common Prefix

Problem: Find longest common prefix among array of strings.

```
public String longestCommonPrefix(String[] strs) {
    if (strs == null || strs.length == 0) return "";

    String prefix = strs[0];
    for (int i = 1; i < strs.length; i++) {
        while (strs[i].indexOf(prefix) != 0) {
            prefix = prefix.substring(0, prefix.length() - 1);
            if (prefix.isEmpty()) return "";
```

```
            }
        }
        return prefix;
    }
```

4. Valid Palindrome

Problem: Check if string is palindrome (alphanumeric only).

```
public boolean isPalindrome(String s) {
    int left = 0, right = s.length() - 1;
    while (left < right) {
        while (left < right && !Character.isLetterOrDigit(s.charAt(left))) left++;
        while (left < right && !Character.isLetterOrDigit(s.charAt(right))) right--;
        if (Character.toLowerCase(s.charAt(left)) != Character.toLowerCase(s.charAt(
            return false;
        }
        left++; right--;
    }
    return true;
}
```

← Previous: Backtracking | Back to Main Guide | Next: Math & Bit Manipulation →